

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет о выполнении лабораторной работы №1
«Лексический анализатор»
по дисциплине «Математическая логика и теория
формальных языков»

Вариант 17

Студент,
группы 5130201/30101

_____ Уэссу д.п.

Преподаватель

_____ Востров А.В.

« ____ » _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	2
1 Математическое описание лексического анализатора	3
1.1 Структура транслятора	3
1.2 Лексический анализ	3
1.3 Назначение лексического анализа	3
1.4 Формальная модель лексического анализатора	4
1.5 Работа лексического анализатора	5
1.6 Синтаксические диаграммы	5
1.7 Обработка ошибок	5
2 Особенности реализации	7
2.1 Структуры данных	7
2.1.1 Класс Лексема	7
2.2 Класс ЛексическийАнализатор	7
2.2.1 Метод получить_номер_идентификатора	8
2.2.2 Метод распознать_комментарий	8
2.2.3 Метод распознать_комплексное	9
2.2.4 Метод распознать_идентификатор	9
2.2.5 Метод анализировать	10
2.3 Вспомогательные функции	12
2.3.1 Функция прочитать_файл	12
2.3.2 Функция main	12
3 Результаты работы программы	13
Заключение	16
Список литературы	17

Введение

Лексический анализ — это первый этап обработки текста программы или входных данных, на котором осуществляется разбор последовательности символов и выделение лексем (токенов) — минимальных значимых единиц языка, таких как ключевые слова, идентификаторы, константы, операторы и разделители.

В данной работе рассматривается разработка программы, выполняющей лексический анализ входного текста в соответствии с заданными требованиями. Основные задачи программы:

- Разбиение входного текста на лексемы.
- Классификация лексем по типам (идентификаторы, константы, ключевые слова и т.д.).
- Проверка ограничений на длину идентификаторов и строковых констант (не более 15 символов).
- Обработка комментариев произвольной длины.
- Вывод сообщений об ошибках, которые могут быть обнаружены на этапе лексического анализа
- Обеспечить поддержку комментариев неограниченной длины во входном файле. Форма записи комментария: `/* ... */`.

Вариант 17: Входной язык содержит только кириллицу, арифметические выражения, разделенные символом ; (точка с запятой). Арифметические выражения состоят из идентификаторов, комплексных чисел (в обычной форме), знака присваивания (`:=`), знаков операций `+`, `-`, `*`, `/`, `^` круглых скобок.

Результатом работы является программа, способная корректно анализировать входной текст, формировать таблицу лексем с указанием их типов и значений, а также сообщать об ошибках, если они присутствуют.

1 Математическое описание лексического анализатора

1.1 Структура транслятора

Транслятор осуществляет преобразование входного текста L в выходные данные. В соответствии с синтаксически-ориентированной трансляцией процесс включает два этапа:

- **Распознаватель** — строит структуру входной цепочки на основе порождающей грамматики входного языка. Данный этап обеспечивает анализ синтаксиса и формирование промежуточной структуры.
- **Генератор** — использует построенную структуру для создания выходных данных, отражающих семантику входной цепочки.

В реальных трансляторах языков программирования этапы могут быть частично совмещены, однако ключевым принципом остаётся построение структуры входных данных (целиком или по частям). На рис.1 представлена схема транслятора рис. 1

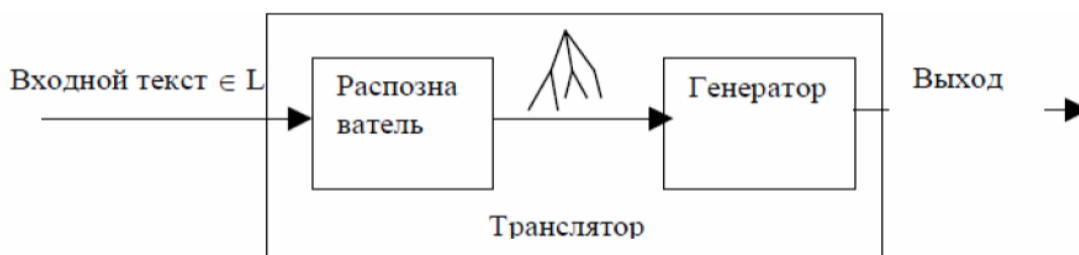


Рис. 1: Структура транслятора

1.2 Лексический анализ

Первая фаза трансляции — лексический анализ, который выделяет лексемы — минимальные единицы языка, обладающие смыслом.

- В естественном языке лексемами являются слова (словоформы).
- В языках программирования — идентификаторы, ключевые слова, константы, составные операторы (например, “:=”).

1.3 Назначение лексического анализа

Лексический анализатор выполняет:

- **Определение значения лексемы** — подстроки символов, соответствующей распознанному классу. Для некоторых классов (например, чисел) значение преобразуется во внутреннее представление (двоичный формат) для компактности и ранней проверки корректности.
- **Определение класса лексемы** — категории элементов с общими свойствами:

- идентификатор,
- Знак присваивания
- Вещественная константа ж
- знак арифметической операции
- круглая скобка.
- разделитель.

1.4 Формальная модель лексического анализатора

Лексический анализатор (лексер) — это конечный автомат, который преобразует входную строку символов в последовательность токенов. Формально его можно описать следующим образом:

Пусть заданы:

- $\Sigma = \{A\text{-}Я, a\text{-}я, 0\text{-}9, :, =, +, -, *, /, \hat{,} (,), ;, \text{space}, \}$ — входной алфавит (множество допустимых символов)
- $T = \{\text{IDENT, COMPLEX, ASSIGN, PLUS, MINUS, MUL, DIV, POW, LPAREN, RPAREN, SEMICOLON}\}$ — множество типов токенов
- $D = D_{\text{ident}} \cup D_{\text{complex}} \cup D_{\text{assign}} \cup D_{\text{op}} \cup D_{\text{bracket}} \cup D_{\text{semicolon}}$
- $D_{\text{ident}} = \{s \mid s \in L \cdot (L \cup N \cup \{_ \})^{k-1}, 1 \leq k \leq 15, L = \{A\text{-}Я, a\text{-}я\}, N = \{0\text{-}9\}\}$ — идентификаторы (начинаются с буквы кириллицы, могут содержать буквы, цифры и символ подчеркивания, длина не более 15 символов);
- $D_{\text{complex}} = \{s \mid s = r + mi \vee s = r - mi \vee s = mi \vee s = -mi, r \in \mathbb{R}, m \in \mathbb{R}\}$ — комплексные числа в форме:
 - 3+4и (действительная и мнимая части);
 - 2.5-1.3и (с плавающей точкой);
 - 7и (чисто мнимое);
 - -5и (отрицательное мнимое);
 - -2-3и (отрицательные части).
- $D_{\text{assign}} = \{:=\}$ — знак присваивания;
- $D_{\text{op}} = \{+, -, *, /, \hat{,}\}$ — арифметические операции;
- $D_{\text{bracket}} = \{(\text{,})\}$ — круглые скобки;
- $D_{\text{semicolon}} = \{;\}$ — разделитель выражений.

Тогда лексический анализатор реализует отображение:

$$F_{\text{lexer}} : \Sigma^* \rightarrow (T \times D)^*$$

Где:

- Σ^* — множество всех возможных строк над алфавитом Σ
- $(T \times D)^*$ — множество последовательностей пар (тип токена, значение)

1.5 Работа лексического анализатора

Процесс лексического анализа можно представить как детерминированный конечный автомат (ДКА):

$$M = (Q, \Sigma, \delta, q_0, F)$$

Где:

- Q — множество состояний автомата
- $\delta : Q \times \Sigma \rightarrow Q$ — функция переходов
- $q_0 \in Q$ — начальное состояние
- $F \subseteq Q$ — множество конечных состояний

Для каждого распознанного токена t_i выполняется:

$$t_i = (\text{type}, \text{value}), \quad \text{где } \text{type} \in T, \text{value} \in D$$

1.6 Синтаксические диаграммы

Синтаксические диаграммы визуализируют правила формирования лексем. На рис.2 рис. 2 представлена диаграмма работы лексического анализатора в данной работе.

y2: Выдать лексему "INDENT, имя"

y3: Выдать лексему "COMPLEX NUMBER, значение"

y4: Выдать лексему ":="

y5: Выдать лексему "^"

y6: Выдать лексему "+"

y7: Выдать лексему "-"

y8: Выдать лексему "*"

y9: Выдать лексему "/"

y10: Выдать лексему "("

y11: Выдать лексему ")"

y12: Выдать лексему ";"

1.7 Обработка ошибок

Лексический анализатор обнаруживает следующие ошибки:

- Недопустимые символы: $\exists c \in w \mid c \notin \Sigma$
- Превышение длины идентификатора: $|id| > 16$
- Незакрытые строковые константы
- Некорректные числовые форматы

Для каждой ошибки генерируется сообщение:

$$E_{\text{lexer}} : \Sigma^* \rightarrow \mathbb{N} \times \text{String}$$

где первое значение — позиция ошибки, второе — описание.

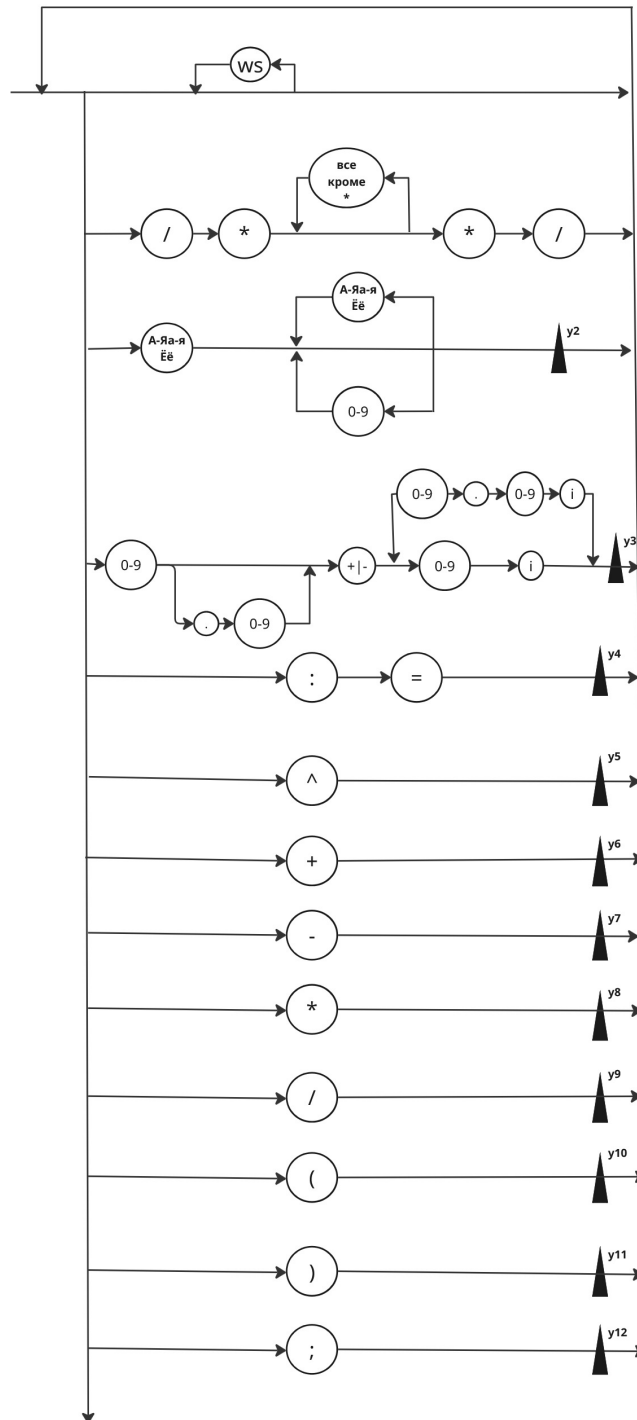


Рис. 2: Сиинтаксическая диаграмма

2 Особенности реализации

Ниже представлено описание ключевых компонентов программной реализации лексического анализатора.

2.1 Структуры данных

2.1.1 Класс Лексема

Класс Лексема предназначен для хранения информации об одной лексеме. Хранение лексемы с её строковым представлением, типом и значением.

- **Вход:** **значение** — исходная строка лексемы, **тип** — категория лексемы (Идентификатор, Число, Оператор), **детали** — дополнительная информация (для идентификаторов — формат "имя:номер").
- **Выход:** Строковое представление лексемы для вывода в таблицу. Метод `__str__` форматирует вывод в виде строки таблицы с фиксированной шириной колонок.

Листинг 1: Класс Лексема

```
1 class лексема:
2     def __init__(self, значение: str, тип: str, детали: str = ""):
3         self.значение = значение
4         self.тип = тип
5         self.детали = детали
6
7     def __str__(self):
8         return f". {self.значение:<20} . {self.тип:<25} . {self.детали:<20} . "
```

2.2 Класс ЛексическийАнализатор

Основной класс, реализующий лексический анализ.

Вход: **текст** — строка, содержащая анализируемый текст.

Выход: Инициализированные поля класса.

- **текст** — входная строка для анализа.
- **позиция** — индекс текущего символа во входной строке.
- **строка, колонка** — координаты текущего символа для локализации ошибок.
- **лексемы** — список распознанных лексем.
- **ошибки** — список сообщений об ошибках.
- **таблица_идентификаторов** — словарь для хранения пар "имя идентификатора: номер".
- **следующий_номер** — счётчик для генерации новых номеров идентификаторов.
- **цифры, операторы, кириллица** — множества символов для быстрой проверки принадлежности.

Листинг 2: Конструктор класса ЛексическийАнализатор

```
1 def __init__(self, текст: str):
2     self.текст = текст
3     self.позиция = 0
4     self.строка = 1
5     self.колонка = 1
6     self.лексемы: List[ексема] = []
7     self.ошибки: List[str] = []
8
9     self.таблица_идентификаторов: Dict[str, int] = {}
10    self.следующий_номер = 1
11
12    self.цифры = set('0123456789')
13    self.операторы = set('+-*/^')
14    self.кириллица = set('абвгдеёжзийклмнопрстуфхцчшщъ ёя
                          ')

```

2.2.1 Метод получить_номер_идентификатора

Вход: **имя** — строковое представление идентификатора.

Выход: Уникальный номер идентификатора.

Реализует таблицу идентификаторов. Если идентификатор встречается впервые, ему присваивается новый номер, который затем сохраняется в словаре. При повторной встрече возвращается уже существующий номер.

Листинг 3: Метод получения номера идентификатора

```
1 def получит_номер_идентификатора(self, имя: str) -> int:
2     if имя in self.таблица_идентификаторов:
3         return self.таблица_идентификаторов[имя]
4     else:
5         номер = self.следующий_номер
6         self.таблица_идентификаторов[имя] = номер
7         self.следующий_номер += 1
8         return номер

```

2.2.2 Метод распознать_комментарий

Вход: Текущая позиция во входном тексте.

Выход: **True** если комментарий распознан, иначе **False**.

Распознаёт комментарии в стиле `/* ... */`. Комментарии полностью игнорируются и не добавляются в таблицу лексем. При обнаружении незакрытого комментария фиксируется ошибка.

Листинг 4: Распознавание комментариев

```
1 def распознат_комментарий(self) -> bool:
2     if (self.текущий_символ() == '/' and self.посмотрет_вперед()
3         == '*'):
4         нач_строка = self.строка
5         нач_колонка = self.колонка
6
7         self.следующий_символ()
8         self.следующий_символ()

```

```

8
9         while self.текущий_символ():
10             if (self.текущий_символ() == '*' and self.посмотрет_вперед() == '/'):
11                 self.следующий_символ()
12                 self.следующий_символ()
13                 return True
14             self.следующий_символ()
15
16         self.ошибки.append(f" трока {нач_строка}, колонка {нач_колонка}: незавершенный комментарий")
17         return True
18     return False

```

2.2.3 Метод распознать_комплексное

Вход: Текущая позиция во входном тексте.

Выход: True если комплексное число распознано, иначе False.

Распознаёт комплексные числа в следующих форматах:

- $a+bi$ — действительная и мнимая части (например, $3+4i$);
- $a-bi$ — с отрицательной мнимой частью ($2.5-1.3i$);
- bi — чисто мнимое число ($7i$);
- $-bi$ — отрицательное мнимое число ($-5i$);
- $-a-bi$ — обе части отрицательные ($-2-3i$).

В случае неудачи выполняется восстановление позиции (*backtracking*).

Листинг 5: Распознавание комплексных чисел

```

1 def распознат_комплексное(self) -> bool:
2     сохран_позиция = self.позиция
3
4     if self.текущий_символ() == 'и':
5
6         self.добавит_лексему(комплекс, " комплексное число", комплекс)
7         return True
8
9     if действ_часть and self.текущий_символ() in '+-':
10         self.добавит_лексему(комплекс, " комплексное число", комплекс)
11         return True
12     self.позиция = сохран_позиция
13     return False

```

2.2.4 Метод распознать_идентификатор

Вход: Текущая позиция во входном тексте.

Выход: True если идентификатор распознан, иначе False.

Распознаёт идентификаторы по правилам:

- Начинаются с буквы кириллицы.
- Могут содержать буквы кириллицы, цифры и символ подчёркивания.
- Максимальная длина — 15 символов.
- Каждому уникальному идентификатору присваивается порядковый номер.

Листинг 6: Распознавание идентификаторов

```

1 def распознат_идентификатор(self) -> bool:
2     начало = self.позиция
3     нач_строка = self.строка
4     нач_колонок = self.колонок
5     if not (self.текущий_символ() and self.текущий_символ() in self
6         .кириллица):
7         return False
8     self.следующий_символ()
9     while (self.текущий_символ() and
10         (self.текущий_символ() in self.кириллица or
11         self.текущий_символ() in self.цифры or
12         self.текущий_символ() == '_')):
13         self.следующий_символ()
14     идентификатор = self.текст[начало:self.позиция]
15     if len(идентификатор) > 15:
16         self.ошибки.append(f" строка {нач_строка}, колонка {нач_колонок}: "
17             f" идентификатор слишком длинный (макс. 15): '{идентификатор}'")
18     return True
19
20     номер = self.получит_номер_идентификатора(идентификатор)
21     значение = f"{идентификатор}:{номер}"
22     self.добавит_лексему(идентификатор, " идентификатор", значение)
23     return True

```

2.2.5 Метод анализировать

Вход: Входной текст, сохранённый в поле `текст`.

Выход: `True` если ошибок нет, иначе `False`. Заполняет списки `лексема` и `ошибки`.

Основной цикл анализа, который последовательно обрабатывает все символы входного текста. На каждом шаге определяется тип текущей лексемы и вызывается соответствующий метод распознавания. Порядок проверок имеет значение:

1. Комментарии — должны быть обработаны в первую очередь.
2. Двухсимвольные операторы (`:=`) — до односимвольных.
3. Односимвольные операторы, скобки, разделители.
4. Числа и комплексные числа.
5. Идентификаторы (только кириллица).

6. Неизвестные символы — ошибка.

Листинг 7: Основной цикл анализа

```
1 def анализироват (self) -> bool:
2     while self.текущий_символ():
3         self.пропустит_пробелы()
4
5         if not self.текущий_символ():
6             break
7
8         текущий = self.текущий_символ()
9         if self.распознат_комментарий():
10             continue
11         if текущий == ':' and self.посмотрет_вперед() == '=':
12             self.добавит_лексему(":", "рисваивание", "")
13             self.следующий_символ()
14             self.следующий_символ()
15             continue
16         if текущий == ';':
17             self.добавит_лексему(";", "азделител", "")
18             self.следующий_символ()
19             continue
20         if текущий == '(':
21             self.добавит_лексему("(", "кобка открывающая", "")
22             self.следующий_символ()
23             continue
24         if текущий == ')':
25             self.добавит_лексему(")", "кобка закрывающая", "")
26             self.следующий_символ()
27             continue
28         if текущий in self.операторы:
29             self.добавит_лексему(текущий, "ператор", "")
30             self.следующий_символ()
31             continue
32         if текущий in self.цифры or текущий == '.' or текущий == '-':
33             if self.распознат_комплексное():
34                 continue
35             число = self.распознат_число()
36             if число:
37                 self.добавит_лексему(число, "исло", число)
38                 continue
39             if текущий in self.цифры:
40                 self.добавит_ошибку(f"екорректное число")
41                 self.следующий_символ()
42                 continue
43         if текущий in self.кириллица:
44             self.распознат_идентификатор()
45             continue
46         self.добавит_ошибку(f"едопустимый символ: '{текущий}'")
47         self.следующий_символ()
48
49     return len(self.ошибки) == 0
```

2.3 Вспомогательные функции

2.3.1 Функция прочитать_файл

Вход: имя_файла — строка с именем файла.

Выход: Содержимое файла в виде строки или пустая строка при ошибке.

Читает файл с кодировкой UTF-8. Обрабатывает исключения при отсутствии файла или других ошибках ввода-вывода.

Листинг 8: Чтение файла

```
1 def прочитать_файл(имя_файла: str) -> str:
2     try:
3         with open(имя_файла, 'r', encoding='utf-8') as f:
4             return f.read()
5     except FileNotFoundError:
6         print(f" айл {имя_файла} не найден")
7         return ""
8     except Exception as e:
9         print(f" ошибка при чтении файла: {e}")
10        return ""
```

2.3.2 Функция main

Вход: Ввод пользователя через консоль.

Выход: Запуск анализа выбранного текста.

Реализует интерактивное меню с тремя режимами работы:

1. Анализ существующего файла.
2. Создание тестовых файлов и последующий анализ.
3. Ввод текста вручную.
4. Выход из программы.

Листинг 9: Главная функция программы

```
1 def main():
2     """ лавная функция программы"""
3     while True:
4         print("
5         print("1.  спол зоват  существующий файл")
6         print("2.  оздат  тестовые файлы")
7         print("3.  вести текст вручную")
8         print("4.  выход")
9
10        выбор = input(" выбор (1-4): ").strip()
11        # ... обработка выбора ...
```

3 Результаты работы программы

Для проверки корректности работы лексического анализатора были подготовлены тестовых файла, содержащих различные сценарии использования.

На рисунке 3 представлен результат анализа файла без ошибок.

```
комплекс := 7и;
выражение := 3.14+2.71и ^ 2;
```

. Лексема	. Тип лексемы	. Значение	.
.....			
. переменная	. Идентификатор	. переменная:1	.
. :=	. Присваивание	.	.
. 3+4и	. Комплексное число	. 3+4и	.
. ;	. Разделитель	.	.
. результат	. Идентификатор	. результат:2	.
. :=	. Присваивание	.	.
. 2.5-1.3и	. Комплексное число	. 2.5-1.3и	.
. *	. Оператор	.	.
. (	. Скобка открывающая	.	.
. комплекс	. Идентификатор	. комплекс:3	.
. +	. Оператор	.	.
. 5и	. Комплексное число	. 5и	.
.)	. Скобка закрывающая	.	.
. ;	. Разделитель	.	.
. комплекс	. Идентификатор	. комплекс:3	.
. :=	. Присваивание	.	.
. 7и	. Комплексное число	. 7и	.
. ;	. Разделитель	.	.
. выражение	. Идентификатор	. выражение:4	.
. :=	. Присваивание	.	.
. 3.14+2.71и	. Комплексное число	. 3.14+2.71и	.
. ^	. Оператор	.	.
. 2	. Комплексное Число	. 2	.
. ;	. Разделитель	.	.
.....			
Анализ завершен без ошибок.			

Рис. 3: Результат анализа test1.txt

На рисунке 4 представлен результат анализа файла, содержащего различные типы ошибок

```
слишкомдлинныйидентификатор := 3+4и;
/*Nombres pas correcte */
A := 3++4и;
И := 3+4j;
var := 5;
```

. Лексема	. Тип лексемы	. Значение	.
.....			
. :=	. Присваивание	.	.
. 3+4и	. Комплексное число	. 3+4и	.
. ;	. Разделитель	.	.
. A	. Идентификатор	. A:1	.
. :=	. Присваивание	.	.
. 3	. Комплексное Число	. 3	.
. +	. Оператор	.	.
. +	. Оператор	.	.
. 4и	. Комплексное число	. 4и	.
. ;	. Разделитель	.	.
. И	. Идентификатор	. И:2	.
. :=	. Присваивание	.	.
. 3	. Комплексное Число	. 3	.
. +	. Оператор	.	.
. ;	. Разделитель	.	.
. :=	. Присваивание	.	.
. 5	. Комплексное Число	. 5	.
. ;	. Разделитель	.	.
.....			
ОБНАРУЖЕНЫ ОШИБКИ			
Строка 2, колонка 1: Идентификатор слишком длинный (макс. 15): 'слишкомдлинныйидентификатор'			
Строка 5, колонка 11: Некорректное число: '4j'			
Строка 6, колонка 1: Недопустимый символ 'v'			
Строка 6, колонка 2: Недопустимый символ 'a'			
Строка 6, колонка 3: Недопустимый символ 'r'			

Рис. 4: Результат анализа test2.txt

На рисунке 5 представлен результат анализа файла, содержащего.

```
Имя файла (например, test1.txt): test3.txt
АНАЛИЗИРУЕМЫЙ ТЕКСТ
г := (2+3и) * (4-1и);
ж := (x+y)^2;
/*No comment *
```

. Лексема	. Тип лексемы	. Значение	.
.....			
. г	. Идентификатор	. г:1	.
. :=	. Присваивание	.	.
. (	. Скобка открывающая	.	.
. 2+3и	. Комплексное число	. 2+3и	.
.)	. Скобка закрывающая	.	.
. *	. Оператор	.	.
. (	. Скобка открывающая	.	.
. 4-1и	. Комплексное число	. 4-1и	.
.)	. Скобка закрывающая	.	.
. ;	. Разделитель	.	.
. ж	. Идентификатор	. ж:2	.
. :=	. Присваивание	.	.
. (	. Скобка открывающая	.	.
. x	. Идентификатор	. x:3	.
. +	. Оператор	.	.
. y	. Идентификатор	. y:4	.
.)	. Скобка закрывающая	.	.
. ^	. Оператор	.	.
. 2	. Комплексное Число	. 2	.
. ;	. Разделитель	.	.
.....			

```
ОБНАРУЖЕНЫ ОШИБКИ
Строка 3, колонка 1: Незавершенный комментарий
```

Рис. 5: Результат анализа test3.txt

Заключение

В ходе выполнения лабораторной работы был разработан лексический анализатор для языка, поддерживающего арифметические выражения с комплексными числами, операторами присваивания, идентификаторами на кириллице, операциями $+$, $-$, $*$, $/$, $^$ и комментариями в стиле `/* ... */`. Входной язык содержит только кириллицу, выражения разделяются символом `;` (точка с запятой). Данный язык является автоматным, так как представим в виде синтаксической диаграммы. Автоматные грамматики — самые простые из формальных грамматик. Они являются контекстно-свободными, но с ограниченными возможностями.

Достоинства реализации

- **Последовательная обработка:** Код читает входные данные последовательно, символ за символом, что позволяет обрабатывать файлы любого размера без необходимости загрузки всего текста в память.
- **Таблица идентификаторов:** Реализована система нумерации идентификаторов, что позволяет однозначно идентифицировать каждый уникальный идентификатор и использовать эту информацию на последующих этапах трансляции.
- **Точная диагностика ошибок:** Все ошибки выводятся с указанием строки и колонки, что упрощает их локализацию и исправление.

Недостатки реализации

- **Ограниченный набор типов данных:** Реализованная версия поддерживает только числа и комплексные числа, что ограничивает область применения анализатора.
- **Нет поддержки многострочных комментариев с вложенностью:** Комментарии не поддерживают вложенность, что является стандартным ограничением для данного типа комментариев.

масштабирование

- **Поддержка вложенных комментариев:** Реализация обработки вложенных комментариев повысит удобство использования языка.
- **Оптимизация распознавания чисел:** Улучшение алгоритмов распознавания чисел для более эффективной обработки и лучшей диагностики ошибок.

Разработанный анализатор может служить основой для компилятора или интерпретатора целевого языка.

Список литературы

- [1] Секция «Телематика». Текст : электронный // URL: <https://tema.spbstu.ru/mathem/> (Дата обращения: 15.03.2026)